

Report Midterm Exercise CNN Lab on hardware-software codesign

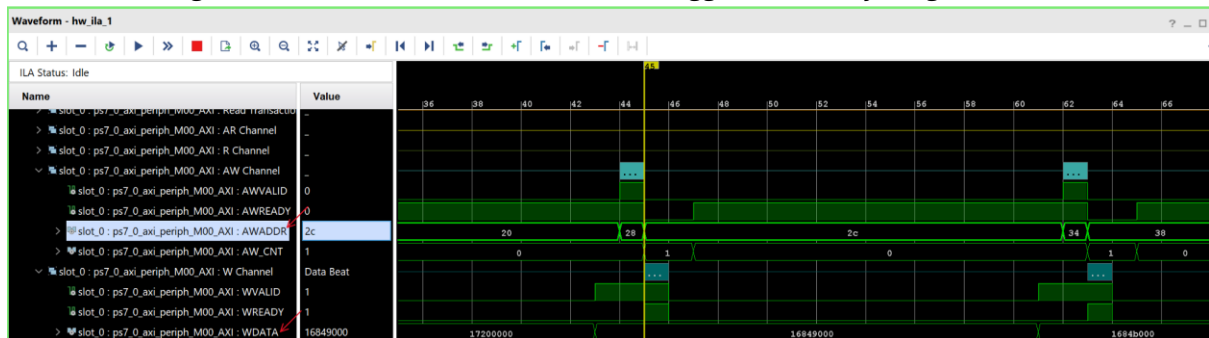
Gaëtan Jenni

Initial Hardware

For the first design, the objective was to just have something fully functional, without any optimizations.

An issue blocked me during more than a week. Everything was working in the C-testbench in Vitis, also in the template to test the 2D convolution. Everything was also correct in my software (allocate DMA memory, etc...). It was therefore hard to find out what was the issue.

I verified the register's addresses with the Vivado debugger, and everything seemed correct:



I finally found that the issue was because I defined the type of the TFXP and TFXP_MULT as unsigned instead of signed in the HLS application.

```
17 typedef uint32_t TFXP; // Parameters and activations
18 typedef uint64_t TFXP_MULT; // Intermediate results of multiplications
```

What is weird is why the template to test the 2D convolution was working. It seems to test only unsigned values, but I didn't invest more time to know why this template was not working.

HW2 - caching filters coefficients

The access to the memory is taking a lot of clock cycle, so if we success to reduce this number to the minimum. To do that a good solution is to retrieve memory data and save in the cache with some buffers.

We can calculate first the number of memory access done by the initial hardware to access the filters coefficients:

$$N_{\text{access}} = \text{numFilters} * \text{numChannels} * \text{inputWidth} * \text{inputHeight} * 9$$

But these filter's coefficients are the same for all the inputWidth and inputHeight, so we can reduce this number of accesses a lot if we cache for each filter all the coefficients needed:

```

34  TFXP tempFilters[MAX_CHANNELS][3][3];
35  for (TFXP iChannel = 0; iChannel < numChannels; ++ iChannel) {
36      #pragma HLS LOOP_TRIPCOUNT min=MAX_CHANNELS max=MAX_CHANNELS
37      for (TFXP cy = 0; cy < convHeight; ++ cy) {
38          #pragma HLS LOOP_TRIPCOUNT min=3 max=3
39          for (TFXP cx = 0; cx < convWidth; ++cx) {
40              #pragma HLS LOOP_TRIPCOUNT min=3 max=3
41              tempFilters[iChannel][cy][cx] = *filters;
42              ++filters;
43          }
44      }
45  }

```

With this modification the number of accesses to the memory for the filter's coefficient is reduced to:

$$N_{access} = numFilters * numChannels * 9$$

We can see in the result array that it as expected also reduces the total time of the CNN application, because the convolution time is a lot reduced.

HW3 - Caching input rows

Then we can do the same things with the input rows. The initial number of accesses is the following:

$$N_{access} = numFilters * numChannels * inputWidth * inputHeight * 9$$

The specificity for the inputs is that we first access to the first 3 rows, and the row 2-3-4, and then 3-4-5, like a moving average. We can therefore reuse each time the 2 last rows instead of reading them in the memory again:

Initialization of the first 3 rows (done for each new filter):

```

35  TFXP tempInput[12192];
36  for (TFXP iChannel = 0; iChannel < numChannels; ++ iChannel) {
37      #pragma HLS LOOP_TRIPCOUNT min=MAX_CHANNELS max=MAX_CHANNELS
38      for (TFXP cy = 0; cy < convHeight; ++ cy) {
39          #pragma HLS LOOP_TRIPCOUNT min=3 max=3
40          for (TFXP x = 0; x < inputWidth; ++x) {
41              #pragma HLS LOOP_TRIPCOUNT min=MAX_WIDTH max=MAX_WIDTH
42              tempInput[iChannel*3*inputWidth + cy*inputWidth + x] = *(input + iChannel*inputHeight*inputWidth + cy*inputWidth + x);
43          }
44      }
45  }

```

The 12192 number is calculated by retrieving his maximum value (calculated in the array of maximum array size further).

Then, each time we need a new row of pixels (so when y increase by one), we retrieve it from the memory, but the 2 other rows remain in cache:

```

79 for (TFXP iChannel = 0; iChannel < numChannels; ++ iChannel) {
80 #pragma HLS LOOP_TRIPCOUNT min=MAX_CHANNELS max=MAX_CHANNELS
81 for (TFXP x = 0; x < inputWidth; ++x) {
82 #pragma HLS LOOP_TRIPCOUNT min=MAX_WIDTH max=MAX_WIDTH
83 tempInput[iChannel*3*inputWidth + iCy*inputWidth+ x] = *(input + iChannel*inputHeight*inputWidth + (y+3)*inputWidth + x);
84 }
85 }
86 iCy += 1;
87 if (iCy == 3) {
88     iCy = 0;
89 }

```

The iCy is there to know at which index is the first row to use.

The number of accesses to the memory for the input is now:

$$N_{\text{access}} = \text{numFilters} * \text{numChannels} * \text{inputWidth} * (3 + \text{inputHeight})$$

The number of accesses to memory is therefore a lot reduced now.

(I noticed later that this was the objective of the point 2.4 : “Optional: Re-use of the caching of the previous rows”, but I directly implemented it by thinking it was already the objective of the point 2.2).

HW4 – Parallelize the computation of the 9 multiply-accumulate

To be able to parallelize the computation of the 9 multiply-accumulate calculation, we need to first partition the 2 arrays where the coefficients and the input values are saved. The type of the partition must be “cyclic” to access the first 9 values in one cycle:

```

27 TFXP tempFilters[MAX_CHANNELS][3][3];
28 #pragma HLS array_partition variable=tempFilters type=cyclic factor=3 dim=2
29 #pragma HLS array_partition variable=tempFilters type=cyclic factor=3 dim=3
30
31 TFXP tempInput[3][4064];
32 #pragma HLS array_partition variable=tempInput type=cyclic factor=3 dim=1

```

Then we can unroll the 2 convolution loop by 3:

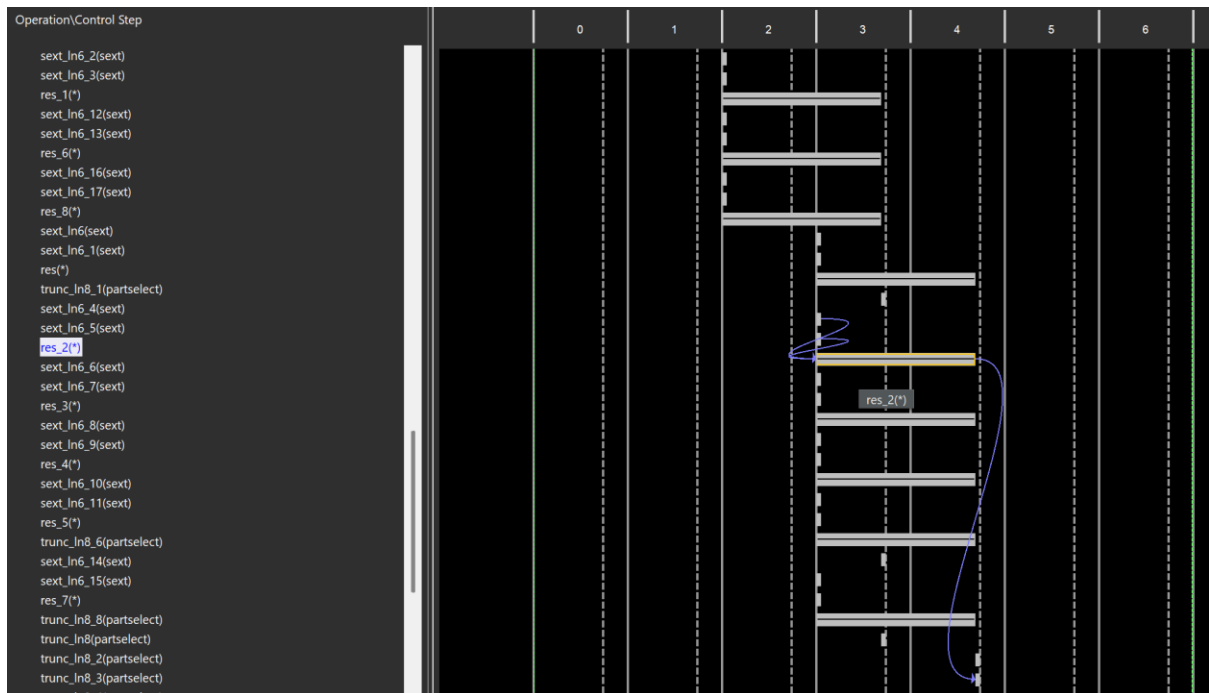
```

62 for (TFXP cy = 0; cy < 3; ++ cy) {
63 #pragma HLS LOOP_TRIPCOUNT min=3 max=3
64 #pragma HLS unroll factor=3
65     TFXP iiCy = (cy+iCy)%3;
66     for (TFXP cx = 0; cx < 3; ++cx) {
67         #pragma HLS LOOP_TRIPCOUNT min=3 max=3
68         #pragma HLS unroll factor=3

```

I noticed that I needed to replace the “convWidth” and “convHeight” variables by 3 to have a constant number of loops. Without it Vitis was not able to unroll the loop.

We can clearly see in the scheduler viewer the parallelization of the 9 multiplications (the multiplication is named by “res”):



The 9 multiplications are not 100% parallel but pipelined, 3 are starting in cycle 2 and the 6 others in cycle 3.

These updates divide the total convolution time by nearly 9, as expected.

HW5 – Parallelizing the computation of output filters

The next step was to implement the parallelization of several output filters. To be able to do that, the filters coefficients need to be stored not only for one filter but for several one:

```

37 for (TFXP iFilter = 0; iFilter < numFilters; iFilter += N_UNROLL_OUTPUT) {
38 #pragma HLS LOOP_TRIPCOUNT min=MAX_FILTERS max=MAX_FILTERS
39   for (TFXP iChannel = 0; iChannel < numChannels; ++ iChannel) {
40     #pragma HLS LOOP_TRIPCOUNT min=MAX_CHANNELS max=MAX_CHANNELS
41     for (TFXP cxy = 0; cxy < 9; ++cxy) {
42       #pragma HLS LOOP_TRIPCOUNT min=9 max=9
43       #pragma HLS unroll factor=9
44       for (TFXP iF = 0; iF < N_UNROLL_OUTPUT; ++iF) {
45         #pragma HLS LOOP_TRIPCOUNT min=N_UNROLL_OUTPUT max=N_UNROLL_OUTPUT
46         #pragma HLS unroll factor=N_UNROLL_OUTPUT
47         tempFilters[iChannel][cxy + iF*9] = *(filters + (iFilter + iF) * numChannels * 9 + iChannel * 9 + cxy);
48       }
49     }

```

We can see that we also change the number of filters to increase at each loop (red arrow).

The partition of the tempFilters array need therefore to be changed to be able to unroll $9*N_UNROLL_OUTPUT$ (which is the number of output filters we want to unroll):

```

27 TFXP tempFilters[MAX_CHANNELS][9*N_UNROLL_OUTPUT];
28 #pragma HLS array_partition variable=tempFilters type=cyclic factor=9*N_UNROLL_OUTPUT dim=2

```

The accumulation variable is therefore also partitioned by the number of unroll output.

```

34 TFXP acc[N_UNROLL_OUTPUT];
35 #pragma HLS array_partition variable=acc type=cyclic factor=N_UNROLL_OUTPUT dim=0

```

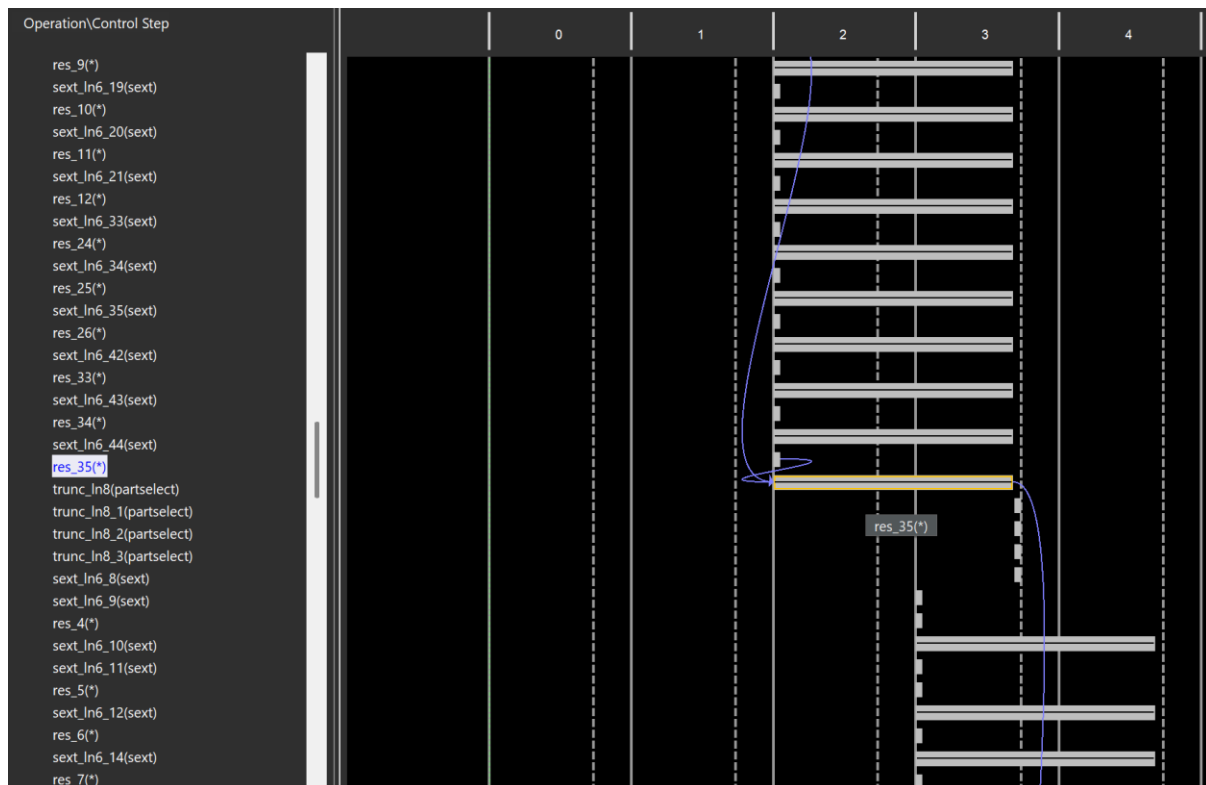
Then we can try to unroll the loop that makes the 9 multiplications:

```

71 for (TFXP cxyf = 0; cxyf < 9*N_UNROLL_OUTPUT; ++cxyf) {
72     #pragma HLS LOOP_TRIPCOUNT min=9*N_UNROLL_OUTPUT max=9*N_UNROLL_OUTPUT
73     #pragma HLS unroll factor=9*N_UNROLL_OUTPUT
74     TFXP cxy = cxyf % 9;
75     TFXP iF = cxyf / 9;
76     TFXP v = tempInput[(cxy / 3 + iCy) % 3][iChannel * inputWidth + x + cxy % 3];
77     acc[iF] += FXP_Mult(tempFilters[iChannel][cxy + iF * 9], v, DECIMALS);
78 }
79 }

```

I defined N_UNROLL_OUTPUT with 4. We can see in the schedule viewer the 9*4 multiplications parallelized but like before everything is done in 3 cycles because Vitis pipelined them.



By testing the hardware, the time only passes from 5.032s to 3.174s, which is far from a division of 4, but the multiplication time seems to be divided by 4.

HW6 – Add the “ReLu” and “AddBiases” functions to the HW

Until now, the “AddBiases” and “ReLu” functions are done in the software. We can accelerate them by doing them directly in HW.

To be able to do that, a new register was added on the AXI bus to transmit the biases to the hardware side. This was simply done by testing if the result of the accumulation is negative (by testing the 32nd bit), and

```
104 acc[iF] += tempBiase[iF];
105 if (acc[iF] & 0x80000000) {
106     acc[iF] = 0;
107 }
108 *(output + (iFilter + iF) * (inputHeight - 2) * (inputWidth - 2) + y * (inputWidth - 2) + x) = acc[iF];
```

A calculation of all maximum array size of the HLS design was done.

inputW	inputH	Channel	Filters	OutputW	OutputH	InputSize	OutputSize	FilterSize	TempFilter	TempInput
256	256	3	32	254	254	196608	2097152	864	27	2304
127	127	32	64	125	125	516128	1032256	18432	288	12192
62	62	64	128	60	60	246016	492032	73728	576	11904
30	30	128	256	28	28	115200	230400	294912	1152	11520
14	14	256	64	12	12	50176	12544	147456	2304	10752
				Maximum array size:		516128	2097152	294912	2304	12192

Each maximum array size was then used to define the depth of the AXI bus inside the HLS design.

HW7 – Overlapping data transfers with computation

Another optimization would be to retrieve in the RAM the next input row needed, in same time than doing the computation. To be able to do that, a function was created to retrieve the next data:

```
11 void load_next_input(TFXP *input, TFXP buffer[4064], TFXP inputWidth, TFXP inputHeight, TFXP numChannels, TFXP y) {
12     #pragma HLS INLINE off
13
14     for (TFXP iChannel = 0; iChannel < numChannels; ++ iChannel) {
15         #pragma HLS LOOP_TRIPCOUNT min=MAX_CHANNELS max=MAX_CHANNELS
16         for (TFXP x = 0; x < inputWidth; ++x) {
17             #pragma HLS LOOP_TRIPCOUNT min=MAX_WIDTH max=MAX_WIDTH
18             buffer[iChannel*inputWidth + x] = *(input + iChannel*inputHeight*inputWidth + y*inputWidth + x);
19         }
20     }
21 }
```

And then inside the loop we call the function with the pragma “DATAFLOW” to parallelize it:

```
83 #pragma HLS DATAFLOW
84 if (y+3 < inputHeight) {
85     load_next_input(input, tempInput[(iCy+3)%4], inputWidth, inputHeight, numChannels, y+3);
86 }
```

Additionally, we need to partition the tempInput array to write and read the

```
42 TFXP tempInput[4][4064];
43 #pragma HLS bind_storage variable=tempInput type=ram_s2p
44 #pragma HLS array_partition variable=tempInput type=cyclic factor=4 dim=1
45 #pragma HLS array_partition variable=tempInput type=cyclic factor=3 dim=2
```

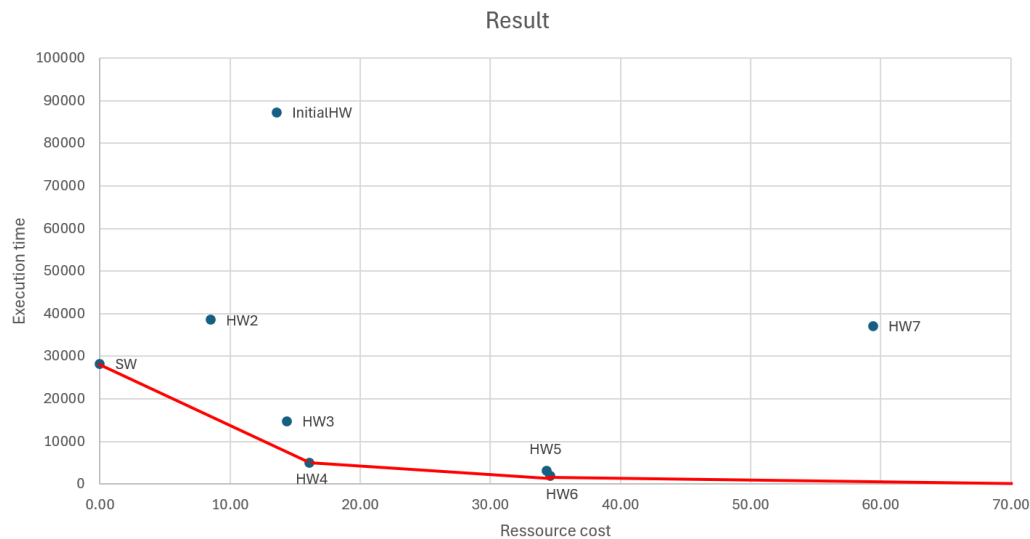
But when I tested with the hardware, the result was correct, but it slowed down a lot (37 seconds). I tried some other things but didn't find what was the issue.

Results and conclusion

Name	Description	Time (ms)	Frequ	LUTs	FFs	BRAMs	DSPs	Cost	Pareto
SW	Pure SW-based app on Pynq	28163	-	0	0	0	0	0.00	Yes
Initial HW	Initial implementation of the accelerator without any optimizations	87175	100	4154	7152	1	86	13.58	No
HW2	Add the caching of filters	38564	100	3975	6053	5	38	8.50	No
HW3	Add the caching of input rows	14737	100	6363	8265	21	50	14.36	No
HW4	Add partitioning and unrolling of loop	5032	100	5465	7501	29.5	57	16.08	Yes
HW5	Parallelizing the computation of output filters (x4)	3174	100	8369	10070	43	179	34.32	No
HW6	Add biases and ReLu function in HW	1881	100	8715	10644	43	179	34.62	Yes
HW7	Overlapping data transfers with computation	37136	100	40685	24784	67	198	59.41	No

The weird number is the 86 DSPs generated by Vivado for the InitialHW. And then for the HW2 only 38 DSPs were synthesized.

There are therefore 3 pareto design: the software, HW4 and HW6:



The final design with the HW is **15** times faster than the pure SW version.